

Getting started with the `abmneo()` function

Sasha D. Hafner

02 September, 2026 10:44

abmneo

The `abmneo` package, with a single `abmneo()` function, is an implementation of the ABM model for anaerobic biodegradation of organic wastes (Dalby et al., 2021). The ABM model is also available in an earlier ABM package. The new package was developed as a simpler but more flexible alternative. Most of that simplicity is in the code; the user experience is roughly similar if default parameter sets are used. For user-defined substrates, microbial groups, or metabolic pathways, the newer package provides much more flexibility. This vignette demonstrates the use of the `abmneo()` model function.

What's different?

The main differences between `abmneo` and ABM are:

1. package complexity and hard-coding,
2. handling of time-variable inputs, and
3. input flexibility for substrates, microbial groups, and stoichiometry.

Not all of these are discussed in this document.

In `abmneo`, time-variable inputs (2 above) are treated using steps instead of the interpolation approach used in ABM. This approach ostensibly reduces the accuracy of the model as a tradeoff for simpler code, more flexibility in which inputs can vary, and faster evaluation. But in practice the effects seem negligible beyond a visual effect in plots.

Hard-coded behavior and special cases in ABM includes the microbial community, with fixed symbols for methanogen and sulfate reducer names and designated code for each. In contrast, the microbial community in `abmneo` is defined completely by input parameters, and stoichiometry can be specified. Internally, shorter vectorized or matrix operations do the work for all groups without special cases. Substrates and inhibition are two other areas where these types of major changes were made.

C++ code and the Rcpp package were incorporated into the ABM package for speed, but at the expense of simple code and easy debugging. So it was dropped from `abmneo`, with any speed penalty apparently offset by other simplifications. In the end `abmneo` is not obviously slower than ABM; it is even slightly faster in some cases. Other dependencies were also eliminated in `abmneo`.

```
devtools::load_all()
```

```
## i Loading abmneo
## Loading required package: deSolve
```

Input arguments and parameters

Here are the `abmneo()` arguments:

```
abmneo <- function(
  storage,
  days = 365,
  delta_t = 1,
  times = NULL,
  inf_pars = NULL,
  grp_pars = NULL,
  sub_pars = NULL,
  chem_pars = list(COD_conv = c(CH4 = 5.32), gases = c('CH4', 'CO2', 'H2S')),
  ctrl_pars = list(
    approx_method = 'early',
    fill_method = 'interp',
    par_key = '\\.',
    h_rate_max_temp = 313
  ),
  var_pars = list(var = NULL),
  add_pars = NULL,
  pars = NULL,
  startup = 0,
  starting = NULL,
  warn = TRUE
)
```

We'll come back to the `storage` argument later. The `days` (total simulation time), `delta_t` (time step *in output*), and `times` (specific times to be included in output) are similar in ABM and straightforward. Let's work on the `*_pars` arguments. These all contain input parameters. Here we will construct a set roughly similar to the ABM set `*_pars_pig2.0` default sets in ABM.

`inf_pars` has influent or inflow properties. VFA is a hard-coded intermediate, and we need to set fresh and initial concentrations (g/kg) along with pH and density. The presence of any other solutes, with any names, is set in the `comps` element, with concentrations in g/kg given in `comp_fresh` and `comp_init` elements. Here the only optional solute we'll add is sulfate, which is needed for sulfate reduction.

```
inf_ref <- list(
  VFA_fresh = 2,
  VFA_init = 2,
  comps = c('S04'),
  comp_fresh = c(S04 = 0.2),
  comp_init = c(S04 = 0.2),
  pH = 7,
  dens = 1000
)
```

`grp_pars` defines the microbial community. The argument should have these elements:

- `grps`: group names
- `yield`: biomass yields on COD basis
- `xa_fresh`: active biomass concentrations in fresh influent
- `xa_init`: initial concentrations
- `d_max`: maximum death/decay rate (1/d)
- `qhat_opt`: maximum substrate utilization rate (g COD / g COD - d)
- `T_opt`: temperature of maximum activity (deg. C)
- `T_min`: minimum temperature with any activity (deg. C)
- `T_max`: maximum temperature with any activity (deg. C)
- `ksmat`: matrix of Monod parameters
- `mstoich`: matrix of stoichiometry

Definition of the microbial community is very flexible; any number of groups with any names and practically any metabolism can be specified. Here we'll include those groups used in the latest ABM parameter set. Let's start with the stoichiometry matrix.

```
mstoich <- matrix(
  c(
    -1, -1, -1, -1, -1, -1, -1,
    0, 0, 0, 0, 0, 0, -0.50156,
    1, 1, 1, 1, 1, 1, 0
  ),
  nrow = 3,
  byrow = TRUE,
  dimnames = list(
    c('VFA', 'S04', 'CH4'),
    c('m0', 'm1', 'm2', 'm3', 'm4', 'm5', 'sr1')
  )
)
```

It is easier to understand by looking at the matrix with row and column names:

```
mstoich

##      m0 m1 m2 m3 m4 m5      sr1
## VFA -1 -1 -1 -1 -1 -1 -1.00000
## S04  0  0  0  0  0  0 -0.50156
## CH4  1  1  1  1  1  1  0.00000
```

Columns are groups, showing the group that carries out the reaction, rows are substrates or products. The values of 1 are an expression of chemical oxygen demand (COD) conservation—they show that each methanogen group produces 1 g of methane per g of VFA consumed (both on a COD basis, when biomass production is neglected). Compounds without a COD are expressed as mass N (if present), S (if present), or otherwise total compound mass. Here then 'S04-2' is as mass of S. So where did the 0.50 come from? The micstoich package was used to sort out the stoichiometry of sulfate reduction.

```
library(micstoich)
rxn <- micstoich('CH3COOH', acceptor = 'S04-2', product = 'H2S')
rxn
molmass('S04-2', 'S') * rxn['S04-2'] / (calcCOD('CH3COOH', per = 'mol') * rxn['CH3COOH'])
```

This takes acetic acid as the representative VFA. And it gives:

```
> library(micstoich)
> rxn <- micstoich('CH3COOH', acceptor = 'S04-2', product = 'H2S')
> rxn
CH3COOH  S04-2      H+      H2S      HS-      H2O      CO2
-0.1250 -0.1250 -0.1875  0.0625  0.0625  0.2500  0.2500
> molmass('S04-2', 'S') * rxn['S04-2'] / (calcCOD('CH3COOH', per = 'mol') * rxn['CH3COOH'])
      S04-2
0.5015625
```

Note that these coefficients do not account for microbial biomass—this information is entered as a microbial yield, and the stoichiometry matrix is updated and expanded internally.

We need one more matrix for `grp_pars`: `ksmat`, which has the Monod half-velocity parameters. Here the values are from `mic_pars2.0` in the ABM repo.

```
ksmat <- matrix(
  c(1.15, 1.15, 1.15, 1.15, 1.15, 1.15, 0.46,
    NA, NA, NA, NA, NA, NA, 0.00694),
  nrow = 7,
  byrow = TRUE,
  dimnames = list(
    c('VFA', 'S04', 'CH4', 'm0', 'm1', 'm2', 'm3'),
    c('m0', 'm1', 'm2', 'm3', 'm4', 'm5', 'sr1')
  )
)
```

```

nrow = 2,
byrow = TRUE,
dimnames = list(
  c('VFA', 'S04'),
  c('m0', 'm1', 'm2', 'm3', 'm4', 'm5', 'sr1')
)
)

ksmat

```

```

##      m0  m1  m2  m3  m4  m5  sr1
## VFA 1.15 1.15 1.15 1.15 1.15 1.15 0.46000
## S04  NA  NA  NA  NA  NA  NA  0.00694

```

Note that NA values for sulfate and the methanogens.

Now let's fill in a `grp_pars` list. Unlike the ABM package, there is no `all` keyword in `abmneo`, but the `default` keyword behaves the same way where no group is explicitly specified.

```

grp_ref <- list(
  grps = c('m0', 'm1', 'm2', 'm3', 'm4', 'm5', 'sr1'),
  yield = c(default = 0.05, sr1 = 0.065),
  xa_fresh = c(default = 0.06),
  xa_init = c(default = 0.1),
  d_max = c(default = 0.02),
  qhat_opt = c(m0 = 0.463, m1 = 0.746, m2 = 0.901, m3 = 2.80, m4 = 4.66, m5 = 7.46, sr1 = 8.34),
  T_opt = c(m0 = 18, m1 = 18, m2 = 28, m3 = 36, m4 = 44, m5 = 55, sr1 = 44),
  T_min = c(m0 = 0, m1 = 9.7, m2 = 9.7, m3 = 15, m4 = 26, m5 = 30, sr1 = 0),
  T_max = c(m0 = 25, m1 = 25, m2 = 38, m3 = 45, m4 = 51, m5 = 60, sr1 = 51),
  ksmat = ksmat,
  mstoich = mstoich
)

```

And check it.

```

grp_ref

## $grps
## [1] "m0" "m1" "m2" "m3" "m4" "m5" "sr1"
##
## $yield
## default sr1
## 0.050 0.065
##
## $xa_fresh
## default
## 0.06
##
## $xa_init
## default
## 0.1
##
## $d_max
## default
## 0.02
##
## $qhat_opt

```

```
##      m0      m1      m2      m3      m4      m5      sr1
## 0.463 0.746 0.901 2.800 4.660 7.460 8.340
##
## $T_opt
##      m0      m1      m2      m3      m4      m5      sr1
##      18      18      28      36      44      55      44
##
## $T_min
##      m0      m1      m2      m3      m4      m5      sr1
##      0.0      9.7      9.7     15.0     26.0     30.0      0.0
##
## $T_max
##      m0      m1      m2      m3      m4      m5      sr1
##      25      25      38      45      51      60      51
##
## $ksmat
##           m0      m1      m2      m3      m4      m5      sr1
## VFA 1.15 1.15 1.15 1.15 1.15 1.15 1.15 0.46000
## SO4   NA   NA   NA   NA   NA   NA   NA 0.00694
##
## $mstoich
##           m0 m1 m2 m3 m4 m5      sr1
## VFA -1 -1 -1 -1 -1 -1 -1.00000
## SO4  0  0  0  0  0  0 -0.50156
## CH4  1  1  1  1  1  1  0.00000
```

Substrates are completely defined in the `sub_pars` input. Like microbial groups, any number and any names could be used. Here we'll continue with something like the ABM parameters. Hydrolysis kinetics follows the same exponential response as in the Arrhenius equation used in ABM, but the parameters are different. In `abmneo`, they are `h_rate_ref`: the rate constant (1/d) at 20 degrees C, and `h_rate_q10`: the relative change in rate over 10 degrees C.

```
sub_ref <- list(
  subs = c('starch', 'Cfat', 'CPs', 'CPf', 'RFd'),
  sub_enrich = c('starch', 'Cfat', 'CPs', 'RFd'),
  xd = 'xd',
  h_rate_ref = c(starch = 0.029, Cfat = 0.058, CPs = 0.012, CPf = 0.012, RFd = 0.0058, xd = 0.0058),
  h_rate_q10 = c(default = 2.1),
  sub_fresh = c(starch = 5.2, Cfat = 28, CPs = 12, CPf = 9.5, RFd = 25, xd = 0),
  sub_init = c(starch = 5.2, Cfat = 28, CPs = 12, CPf = 9.5, RFd = 25, xd = 0)
)
```

So now we've created lists for `inf_pars`, `grp_pars`, and `sub_pars`. These are the only required parameter groups.

The `chem_pars` argument has COD conversion ratios in g COD per g output or input units. These are needed for the COD balance and expressing output in useful units. The default element is for methane, as g COD per g methane C. VFA and substrates are all in COD units anyway, so there is no conversion. The `gases` element names the components that should be treated as gases; they do not accumulate in the slurry. Cumulative emission and emission rate is calculated for each gas. All other components are solutes, which accumulate until the storage is emptied.

The `var_pars` argument is used for time-variable inputs. We won't use it in the first example. And `add_pars` is for adding or modifying parameters set through other arguments. We won't use that in this first example either.

But we do need to sort out the `storage` argument. There are two basic approaches that can be used:

“regular”, with a fixed rate of influent addition and a fixed emptying interval; and “series”, where influent addition and storage emptying is completely flexible, and defined by a slurry level over time.

Regular option

For the regular option, `abmneo()` needs a list with these elements:

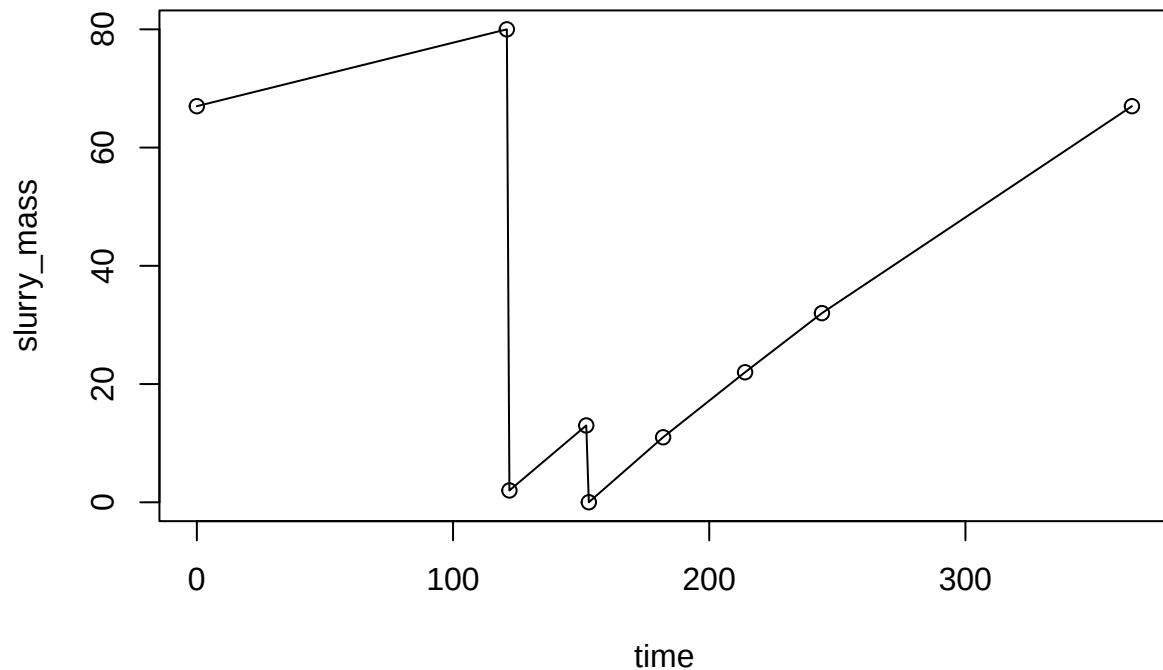
```
stor_reg_ref <- list(  
  type = 'regular',  
  slurry_prod_rate = 5,  
  storage_depth = 2,  
  area = 0.65,  
  temp_C = 20,  
  resid_enrich = 0.6,  
  slurry_mass = 7,  
  resid_depth = 0.01,  
  empty_int = 28,  
  wash_water = 0,  
  wash_int = 90,  
  rest_d = 1  
)
```

These are approximate values for slurry in a pig house with monthly emptying, with values for one pig. Units here are kg for mass, m for length, and d for time.

Series option

And for a series, the main element is a data frame with the measurements.

```
mass_series <- data.frame(  
  time = c(0, 121, 122, 152, 153, 182, 214, 244, 365),  
  slurry_mass = c(67, 80, 2, 13, 0.01, 11, 22, 32, 67)  
)  
plot(slurry_mass ~ time, data = mass_series, type = 'o')
```



```
stor_ser_ref <- list(
  type = 'series',
  dat = mass_series,
  storage_depth = 2,
  area = 0.7,
  temp_C = 18,
  resid_enrich = 0.6
)
```

This is for an outdoor storage tank, with values for single pig.

Running the model

Finally, let's run the model, first with the regular option.

```
out_reg <- abmneo(
  storage = stor_reg_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref
)
```

Output is a data frame that contains gas emission rates, cumulative emission, and masses and concentrations of microbial biomass, substrates, VFA, and any other solutes. The column names, and number of columns, depends on the inputs; each microbial group and each substrate has two associated columns (total mass in g COD and concentration in g COD / kg).

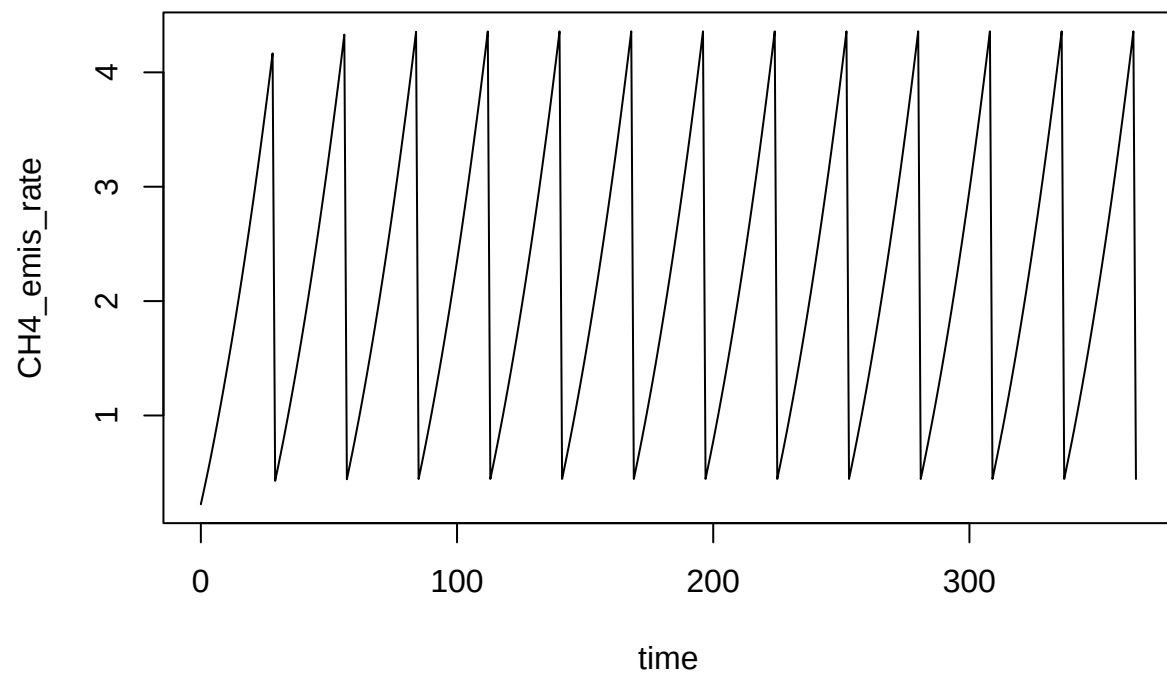
```
names(out_reg)
```

```
## [1] "time"          "m0"            "m1"
## [4] "m2"            "m3"            "m4"
## [7] "m5"            "sr1"           "starch"
## [10] "Cfat"          "CPs"           "CPf"
## [13] "RFd"           "xd"            "VFA"
## [16] "S04"           "slurry_mass"   "CH4"
## [19] "slurry_load"   "COD_load"      "CH4_emis_rate"
## [22] "temp_C"        "pH"            "m0_eff"
## [25] "m1_eff"        "m2_eff"        "m3_eff"
## [28] "m4_eff"        "m5_eff"        "sr1_eff"
## [31] "starch_eff"    "Cfat_eff"      "CPs_eff"
## [34] "CPf_eff"       "RFd_eff"       "xd_eff"
## [37] "VFA_eff"       "S04_eff"       "slurry_mass_eff"
## [40] "slurry_depth"  "m0_conc"       "m1_conc"
## [43] "m2_conc"       "m3_conc"       "m4_conc"
## [46] "m5_conc"       "sr1_conc"      "starch_conc"
## [49] "Cfat_conc"     "CPs_conc"      "CPf_conc"
## [52] "RFd_conc"     "xd_conc"       "VFA_conc"
## [55] "S04_conc"     "m0_eff_conc"   "m1_eff_conc"
## [58] "m2_eff_conc"   "m3_eff_conc"   "m4_eff_conc"
## [61] "m5_eff_conc"   "sr1_eff_conc"   "starch_eff_conc"
## [64] "Cfat_eff_conc" "CPs_eff_conc"   "CPf_eff_conc"
## [67] "RFd_eff_conc"  "xd_eff_conc"   "VFA_eff_conc"
## [70] "S04_eff_conc"  "CH4_emis_rate_ave"
```

The plots below show some results.

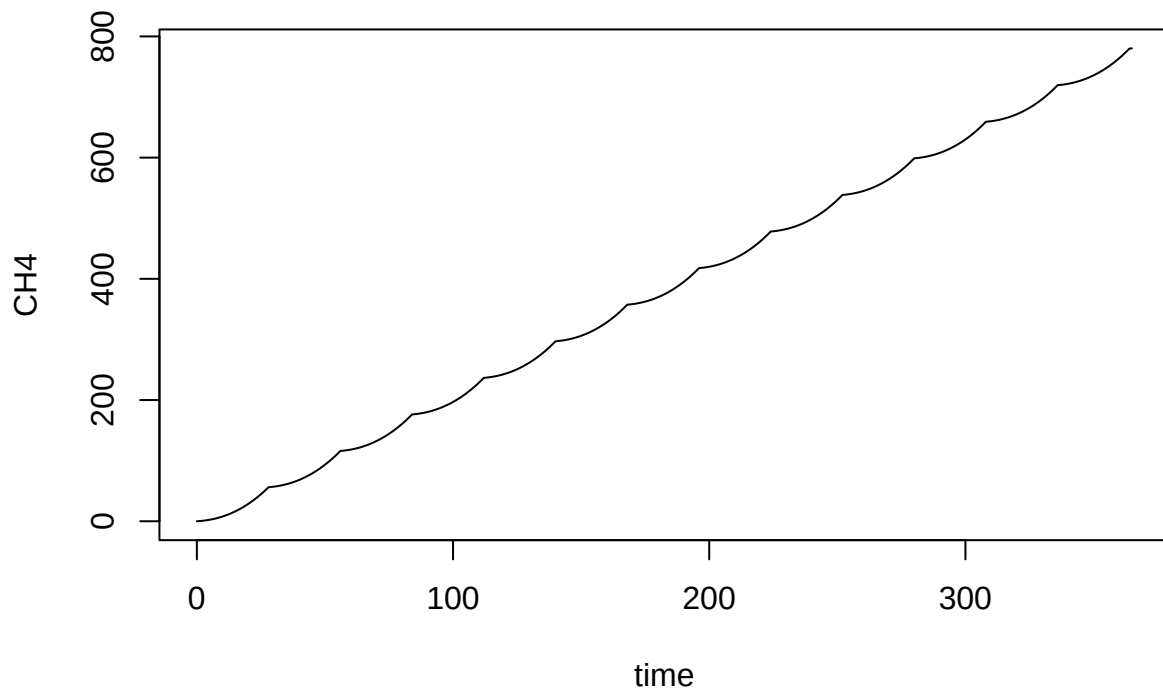
Methane emission rate, in g/d.

```
plot(CH4_emis_rate ~ time, data = out_reg, type = 'l')
```



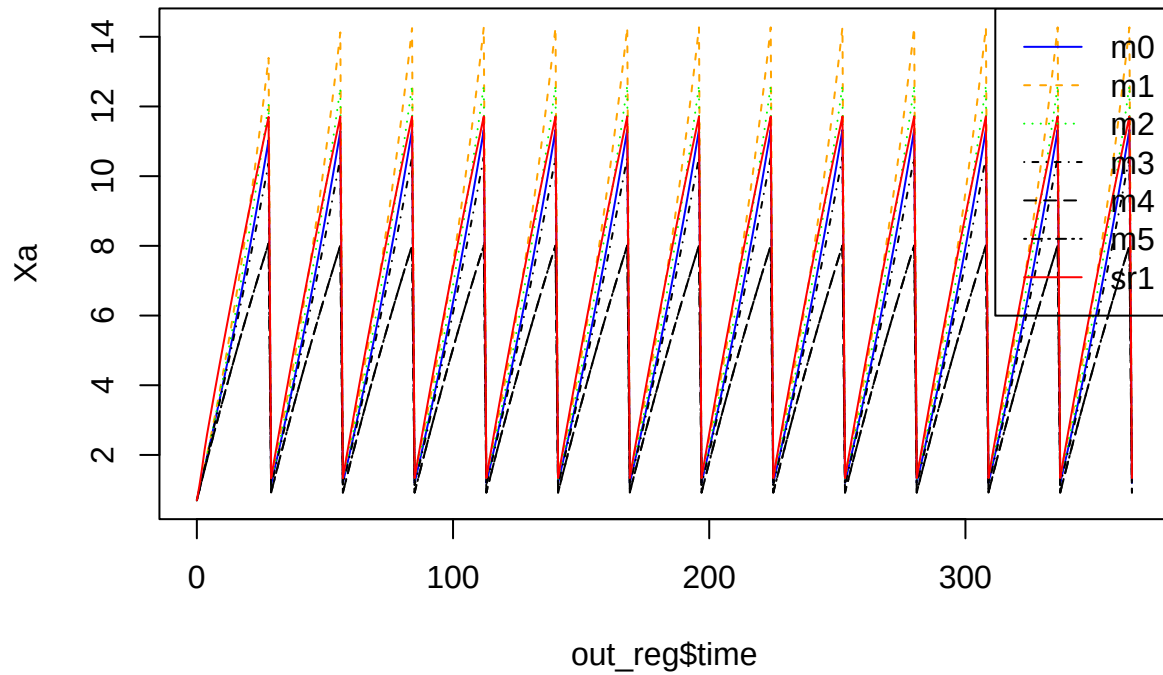
And cumulative emission in g.

```
plot(CH4 ~ time, data = out_reg, type = 'l')
```



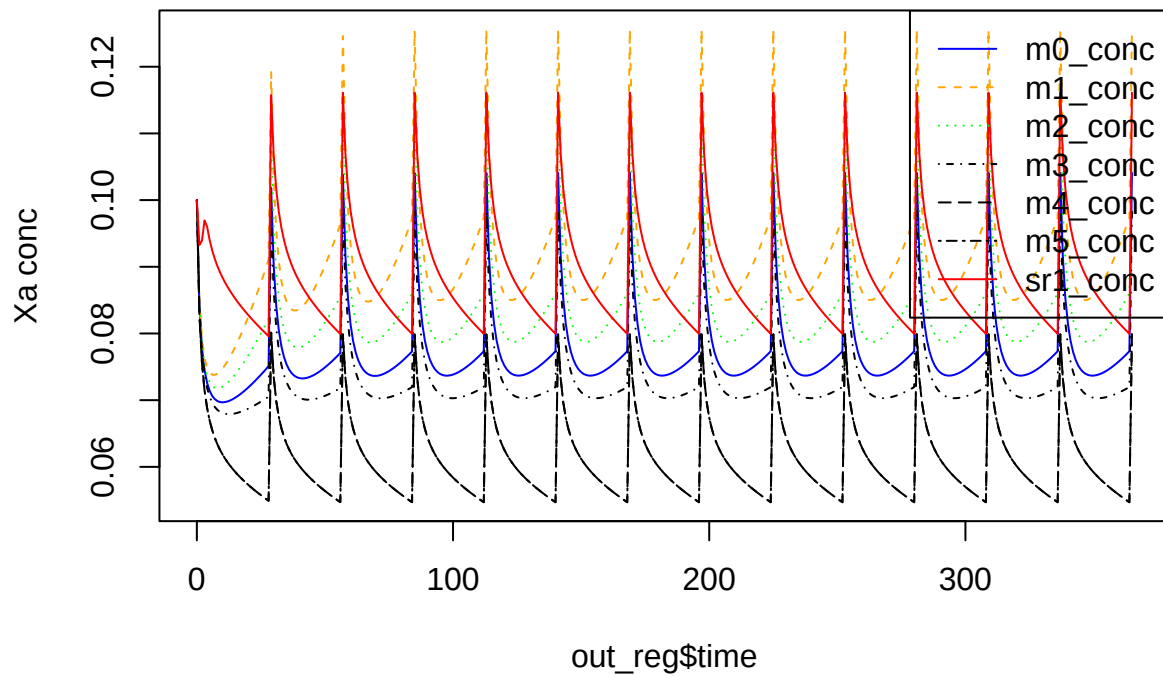
Microbial biomass (g COD).

```
matplot(
  out_reg$time,
  out_reg[, nn <- grp_ref$grps],
  col = cc <- c('blue', 'orange', 'green', 'black', 'black', 'black', 'red'),
  lty = 1:length(nn), type = 'l',
  ylab = 'Xa'
)
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```



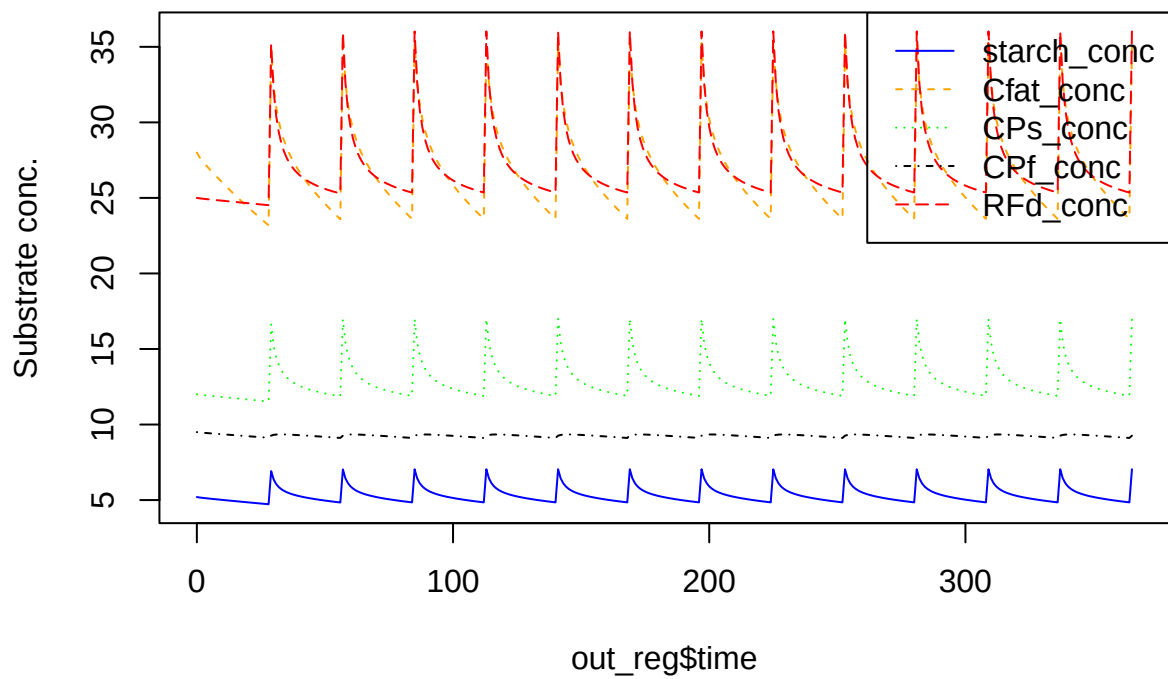
And microbial biomass concentrations (g COD / kg).

```
matplot(
  out_reg$time,
  out_reg[, nn <- paste0(grp_ref$grps, '_conc')],
  col = cc <- c('blue', 'orange', 'green', 'black', 'black', 'black', 'red'),
  lty = 1:length(nn),
  type = 'l',
  ylab = 'Xa conc'
)
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```



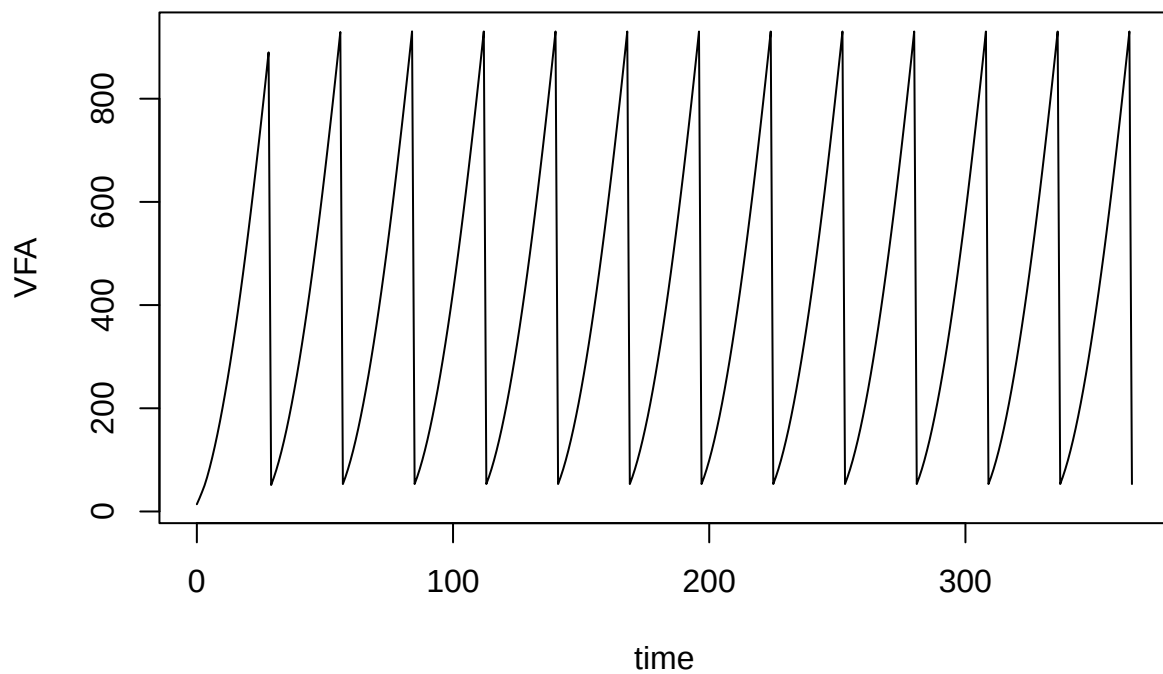
Substrate concentrations.

```
matplot(
  out_reg$time,
  out_reg[, nn <- paste0(sub_ref$subs, '_conc')],
  col = cc <- c('blue', 'orange', 'green', 'black', 'red'),
  lty = 1:length(nn),
  type = 'l',
  ylab = 'Substrate conc.'
)
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```

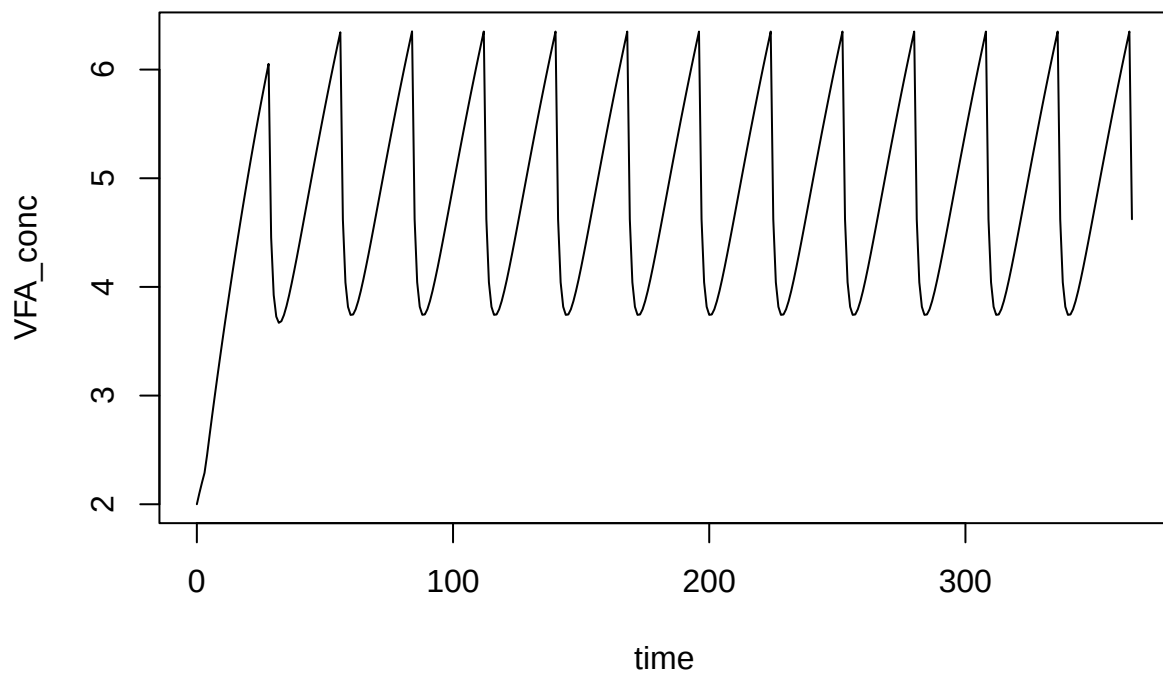


And VFA mass (g COD) and concentration (g COD / kg).

```
plot(VFA ~ time, data = out_reg, type = 'l')
```

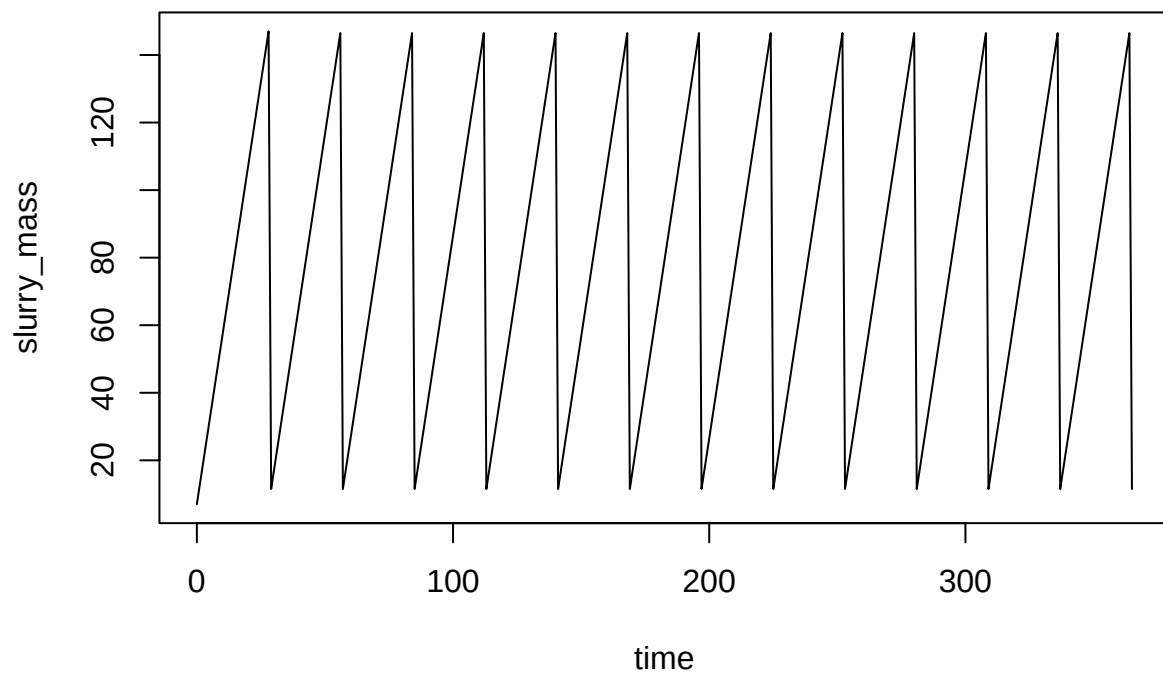


```
plot(VFA_conc ~ time, data = out_reg, type = 'l')
```



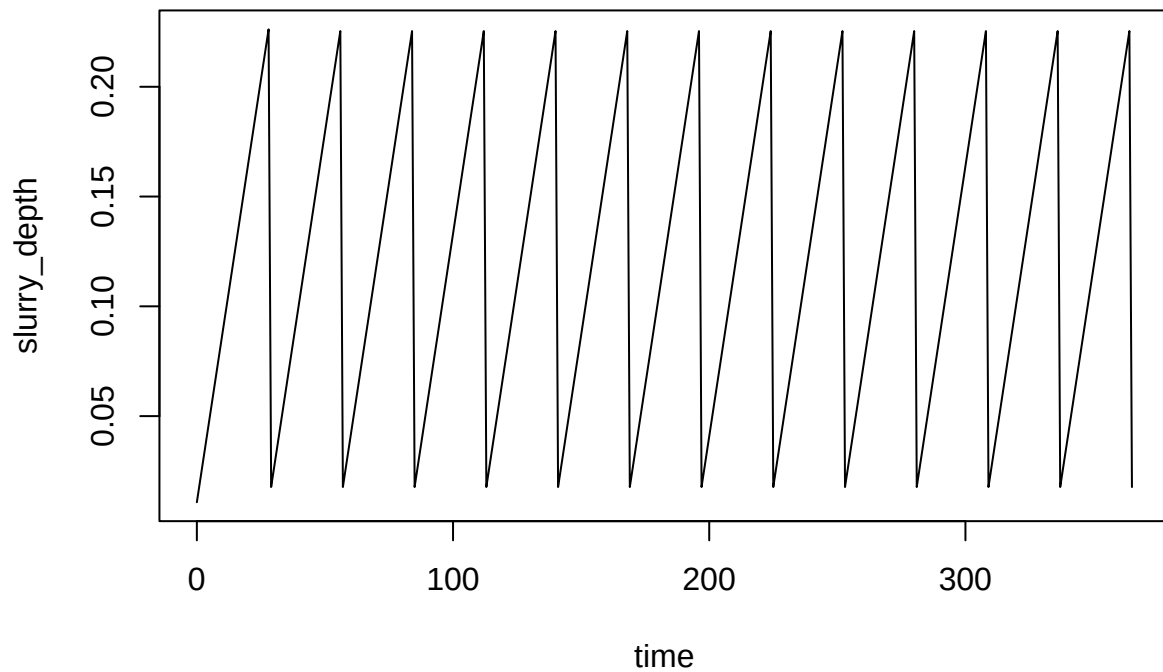
Of course most of the temporal patterns are driven by slurry mass, which is also returned.

```
plot(slurry_mass ~ time, data = out_reg, type = 'l')
```



And so is slurry depth (m).

```
plot(slurry_depth ~ time, data = out_reg, type = 'l')
```

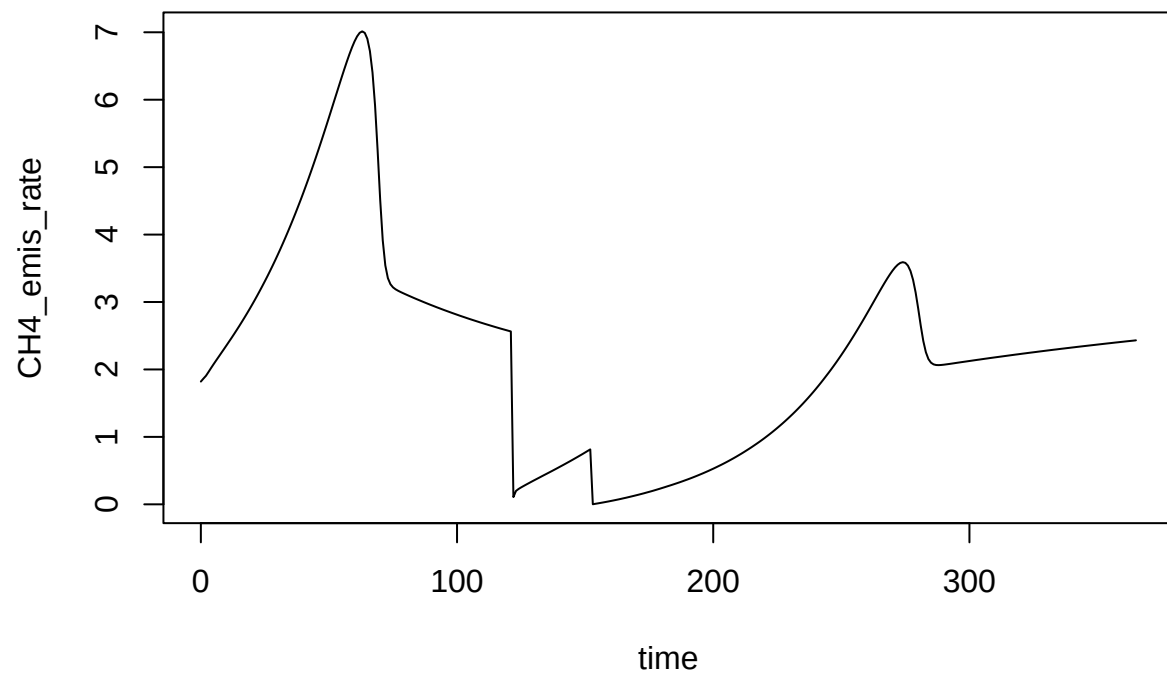


And the series input option.

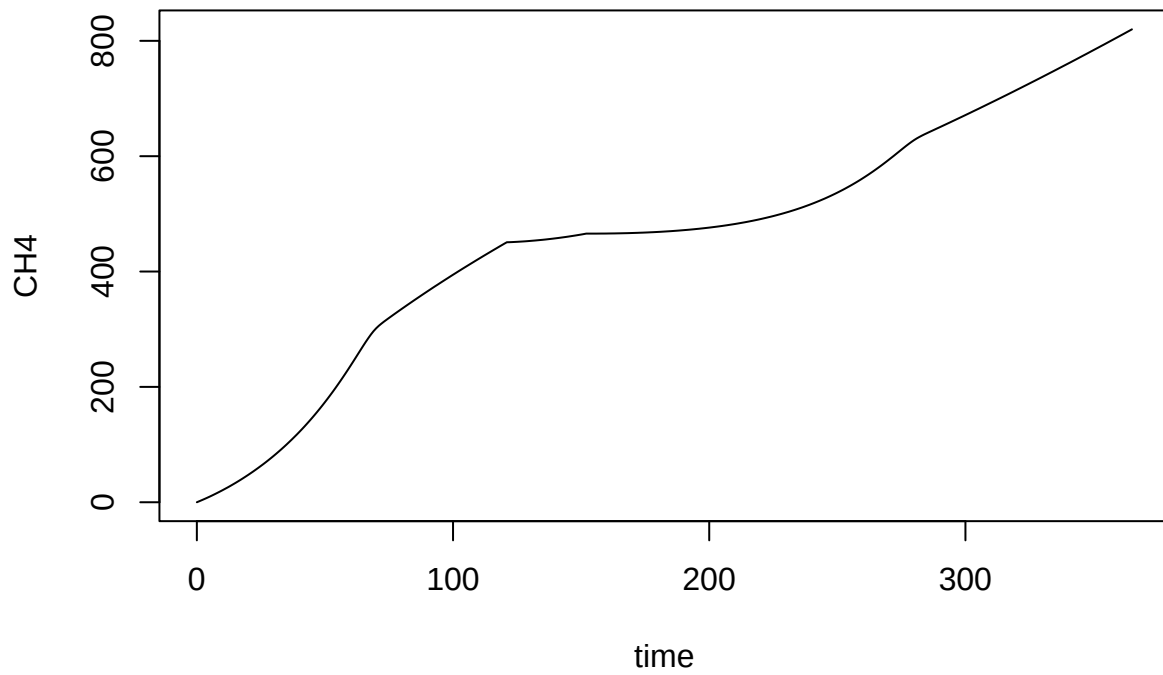
```
out_ser <- abmneo(
  storage = stor_ser_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref
)
```

Here are the same plots that were shown above for regular inputs.

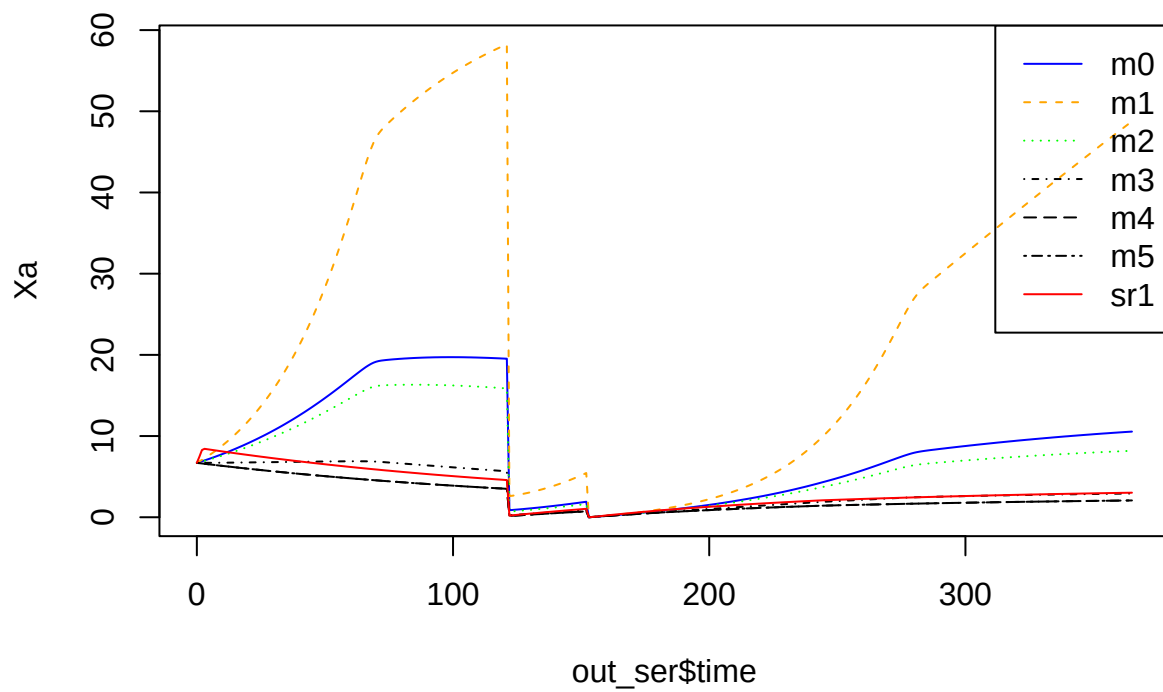
```
plot(CH4_emis_rate ~ time, data = out_ser, type = 'l')
```



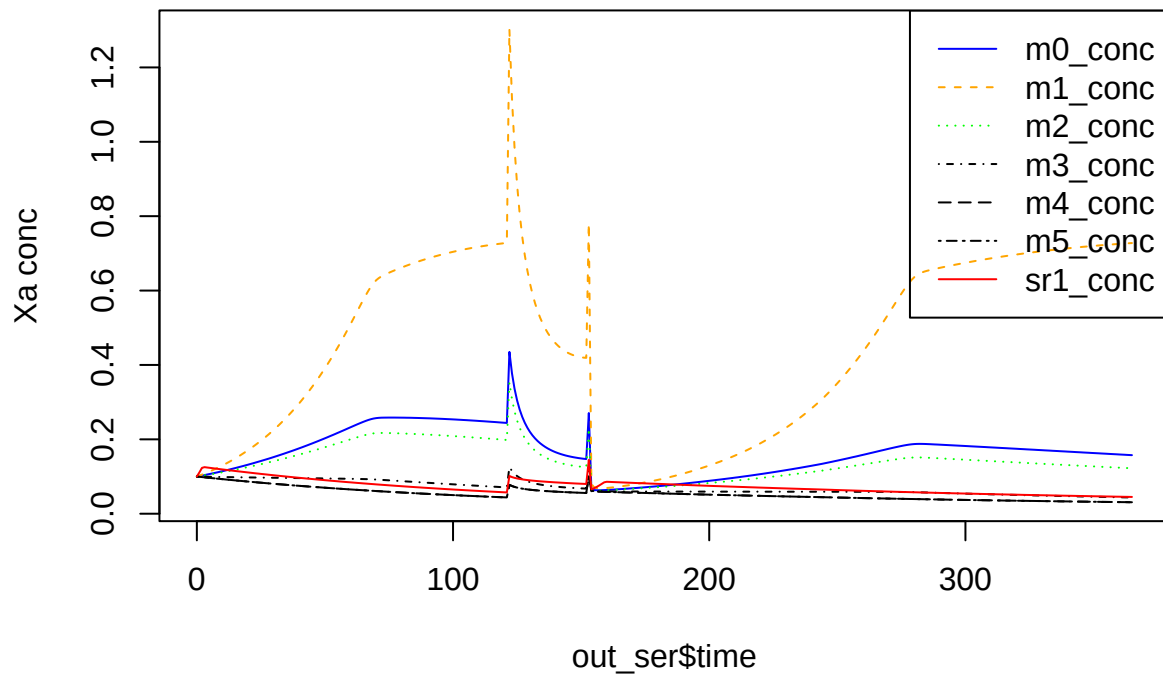
```
plot(CH4 ~ time, data = out_ser, type = 'l')
```



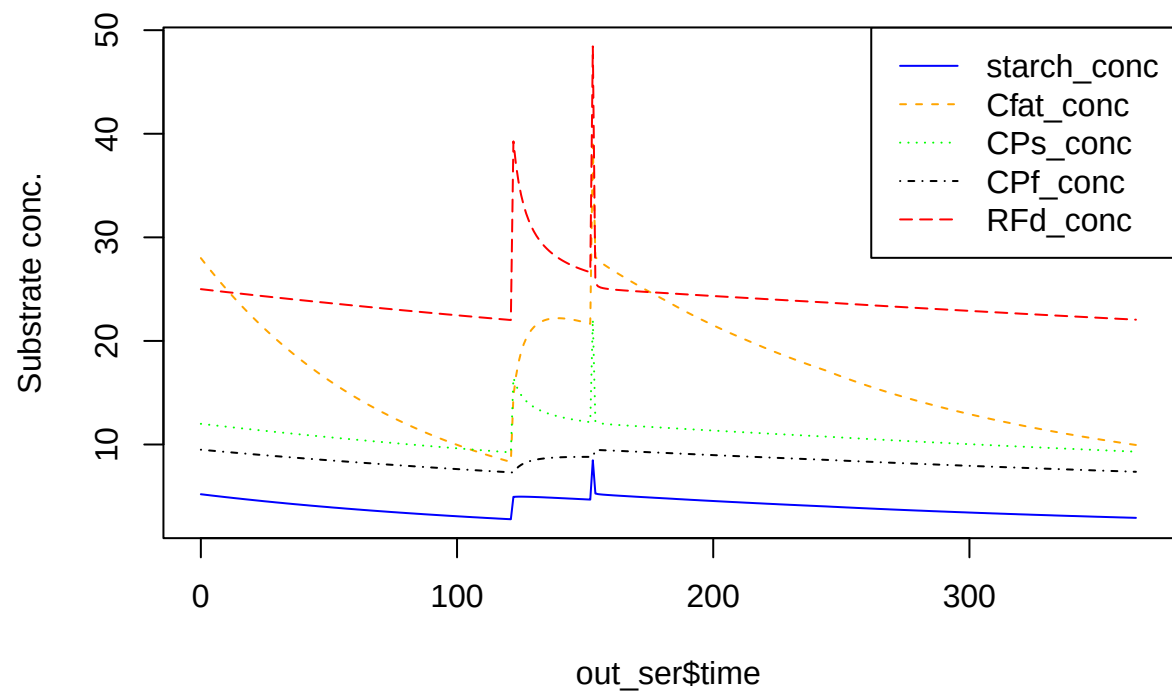
```
matplot(
  out_ser$time,
  out_ser[, nn <- grp_ref$grps],
  col = cc <- c('blue', 'orange', 'green', 'black', 'black', 'black', 'red'),
  lty = 1:length(nn), type = 'l',
  ylab = 'Xa'
)
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```



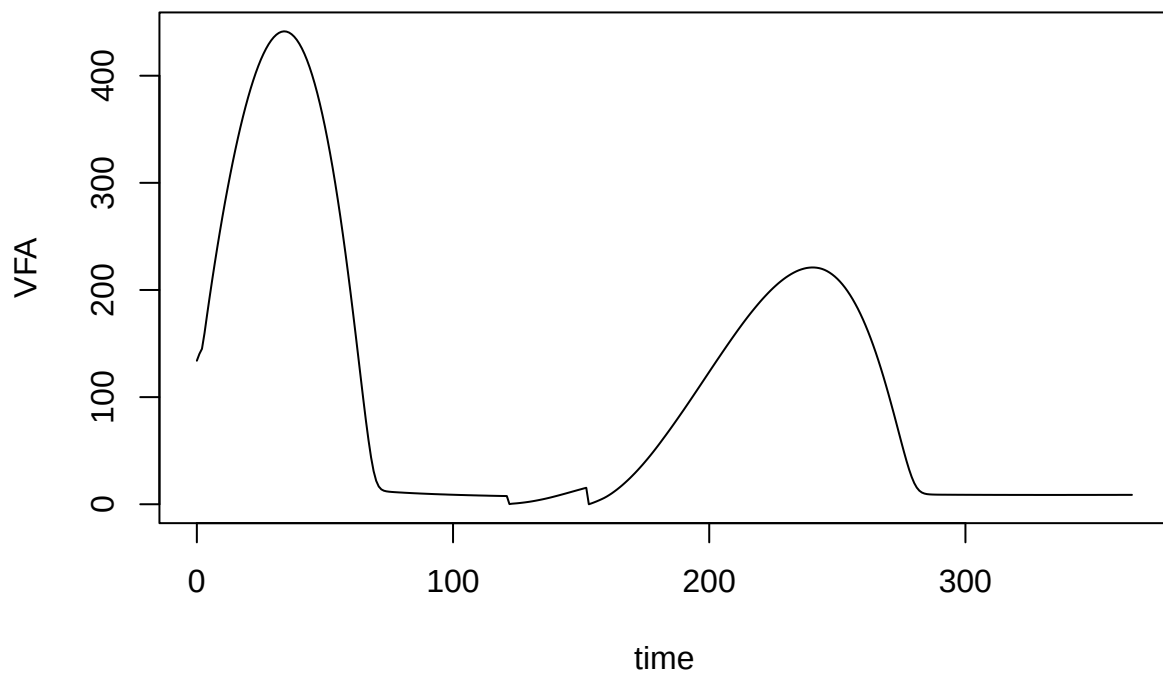
```
matplot(
  out_ser$time,
  out_ser[, nn <- paste0(grp_ref$grps, '_conc')],
  col = cc <- c('blue', 'orange', 'green', 'black', 'black', 'black', 'red'),
  lty = 1:length(nn),
  type = 'l',
  ylab = 'Xa conc'
)
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```



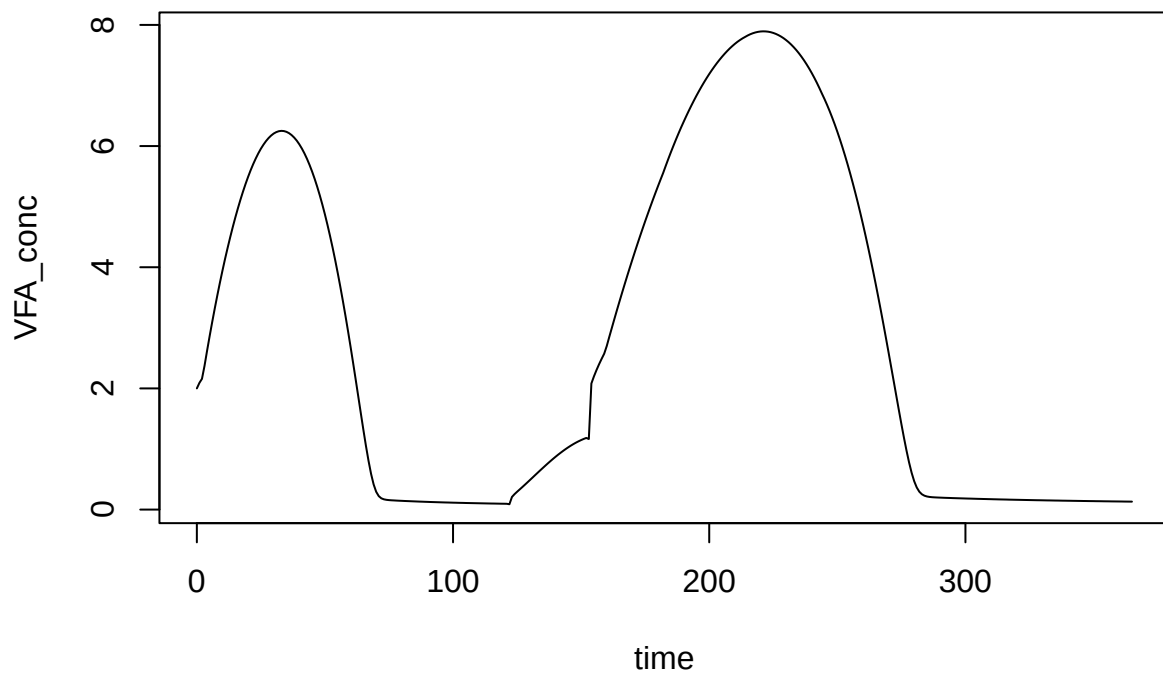
```
matplot(
  out_ser$time,
  out_ser[, nn <- paste0(sub_ref$subs, '_conc')],
  col = cc <- c('blue', 'orange', 'green', 'black', 'red'),
  lty = 1:length(nn),
  type = 'l',
  ylab = 'Substrate conc.'
)
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```



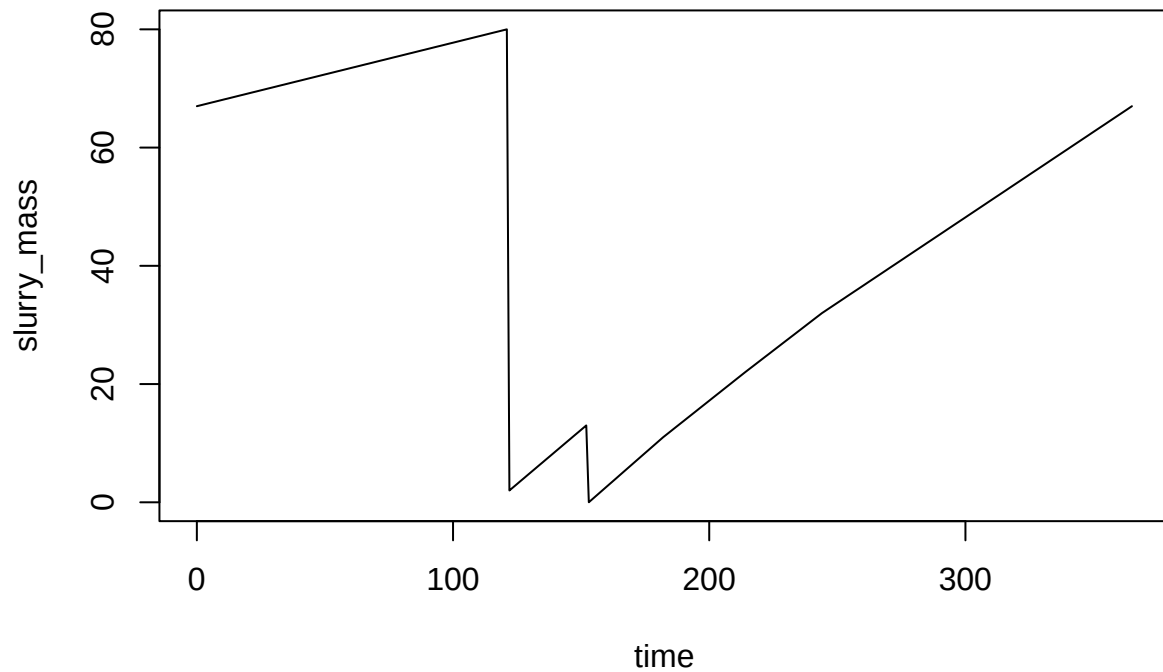
```
plot(VFA ~ time, data = out_ser, type = 'l')
```



```
plot(VFA_conc ~ time, data = out_ser, type = 'l')
```



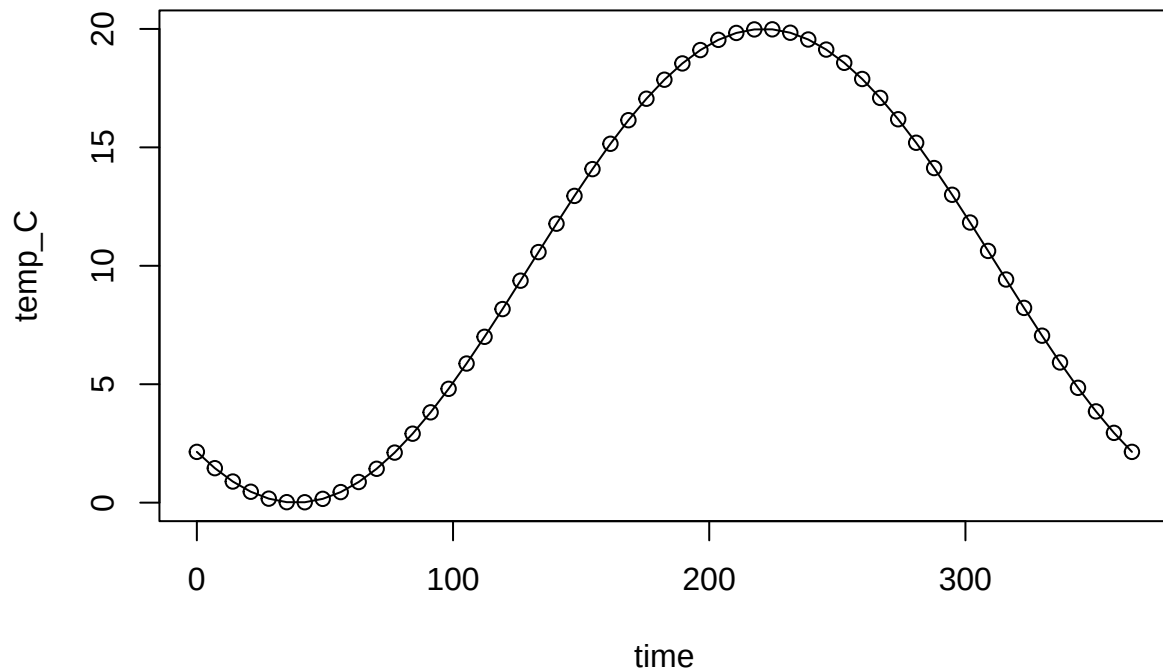
```
plot(slurry_mass ~ time, data = out_ser, type = 'l')
```



Time-variable inputs

Most inputs can be varied over time. They go into the `var_pars` argument as named data frames in a list. Here is an example with temperature.

```
temp_dat <- data.frame(time = seq(0, 365, length.out = 53))
temp_dat$temp_C <- 10 + 10 * sin((temp_dat$time - 130) / 365 * 2 * pi)
plot(temp_C ~ time, data = temp_dat, type = 'o')
```

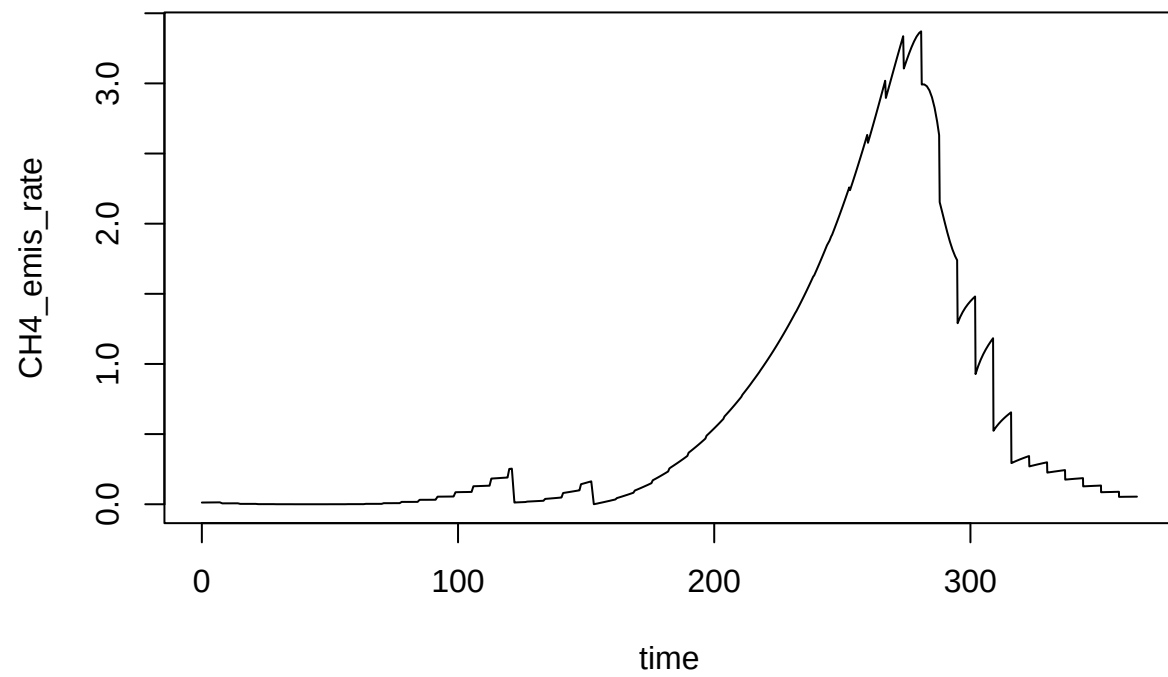


```
out_var <- abmneo(
  storage = stor_ser_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  var_pars = list(var = temp_dat),
)
```

And some of the same plots again.

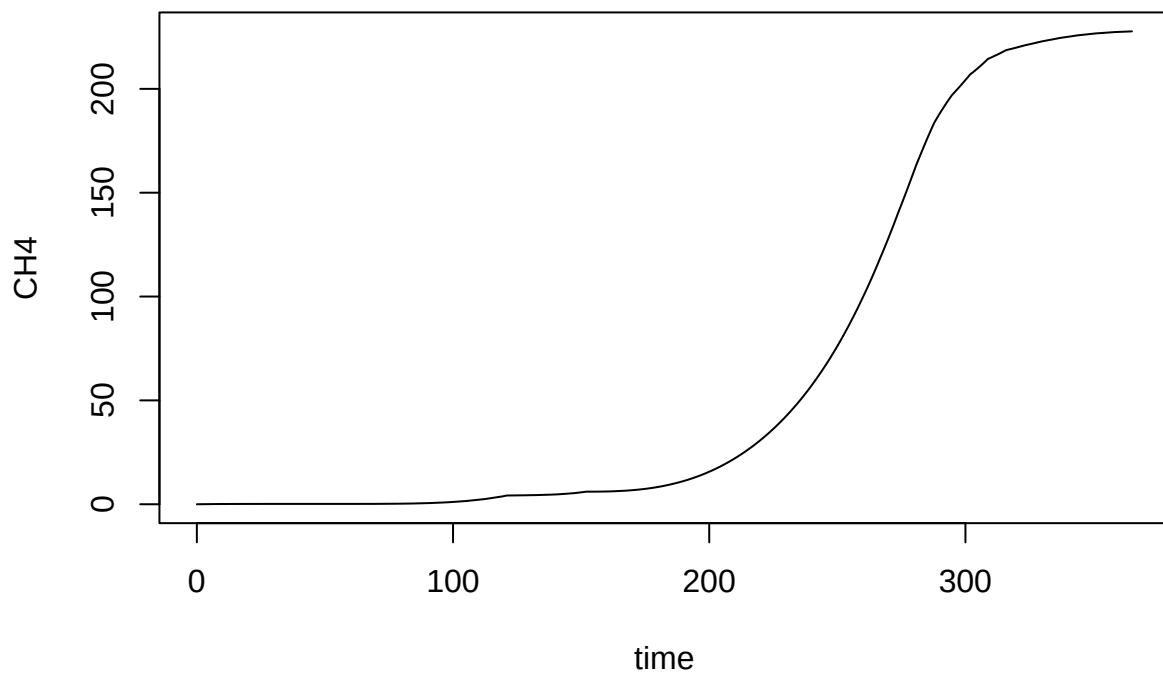
Emission rate.

```
plot(CH4_emis_rate ~ time, data = out_var, type = 'l')
```



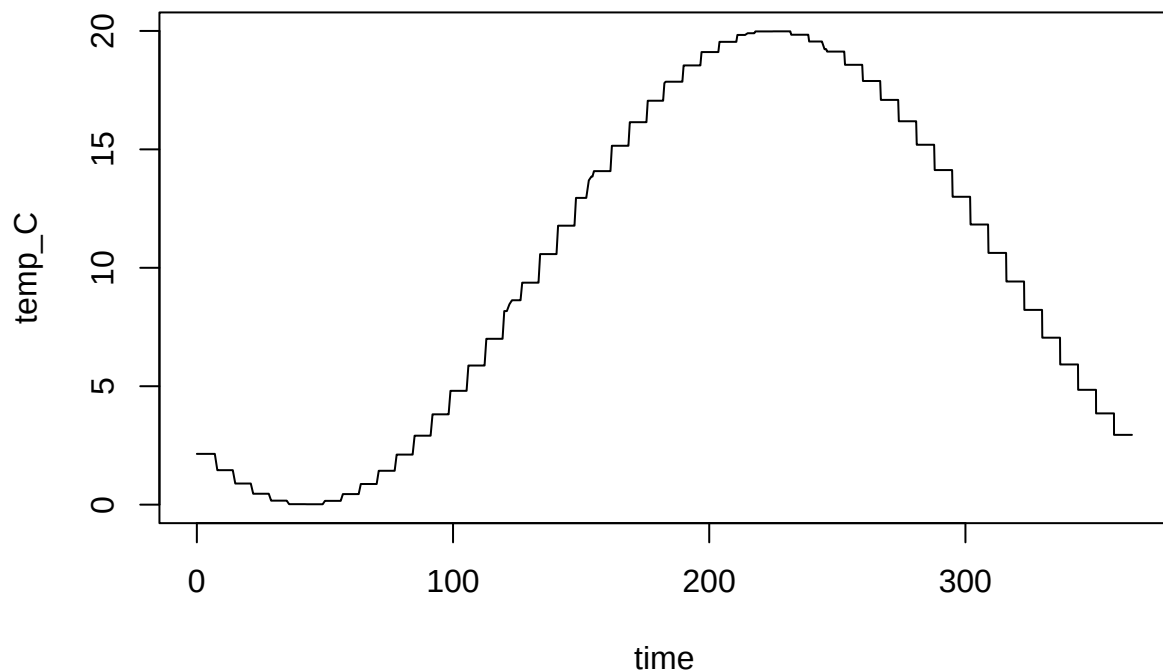
Cumulative emission.

```
plot(CH4 ~ time, data = out_var, type = 'l')
```



Temperature is also returned.

```
plot(temp_C ~ time, data = out_var, type = 'l')
```



Here you can see how time-varying inputs are handled. Instead of smooth interpolation, they are held constant within the steps implied in the `var_pars` input.

References

Dalby, F.R., Hafner, S.D., Petersen, S.O., Vanderzaag, A., Habtewold, J., Dunfield, K., Chantigny, M.H., Sommer, S.G., 2021. A mechanistic model of methane emission from animal slurry with a focus on microbial groups. PLOS ONE 16, e0252881. [10.1371/journal.pone.0252881](https://doi.org/10.1371/journal.pone.0252881)